GO / PDF GENERATION

Generate PDFs from HTML and Go components

A precise, testable document engine for production Go services.

Render HTML in one call or compose explicit PDF pages from rows, columns, and reusable components.

```
$ go get github.com/avdoseferovic/paper
```

Invoice

NO. INV-2026-0481

BILLED TO	ISSUED
Northwind Trading Co.	June 8, 2026

ITEM	AMOUNT
Document API - Team	$480.00
Rendering credits	$240.00
TOTAL DUE	$840.00

main.go - one call

```
package main

import "github.com/avdoseferovic/paper"

func main() {
pdf, _ := paper.FromHTML(context.Background(), invoiceHTML)
  _ = pdf.Save("invoice.pdf")
}
```

layout.go - components

```
doc := paper.New(cfg)
doc.AddRows(row.New(12).Add(
  col.New(8).Add(title),
  col.New(4).Add(qrCode),
))
pdf, _ := doc.Generate(context.Background())
```

Two Authoring Paths

Start from HTML in one call, or compose deterministic documents with rows, columns, and typed components.

HTML -> PDF

Bring existing templates and fragments. Paper translates supported HTML and CSS into a print-ready PDF document.

```
pdf, err := paper.FromHTML(context.Background(), tmpl)
if err != nil { return err }
_ = pdf.Save("invoice.pdf")
```

templates fragments CSS-aware

Component grid

Build explicit document layouts with a 12-column grid. Every primitive is inspectable and reusable in tests.

```
m.AddRows(row.New(12).Add(
  col.New(6).Add(header),
  col.New(6).Add(total),
))
```

12 columns typed testable

.Save()	.Bytes()	.Base64()	.Merge()	+ HTML fragments
write to disk	raw bytes	API payloads	combine PDFs	mix markup inside the component grid

Component Library

Every document primitive can be dropped into a column, composed into larger blocks, and inspected in tests.

Text

Headings, labels, and body copy wrap inside the available column width.

Rich text

Mix bold, links, and accents.

Images

Tables

Item	Qty	Total
API	1	$480
Credits	12k	$240

QR codes

Barcodes

Data Matrix

Checkboxes

☒ Approved

☐ Pending

Signatures

Avdo S.

Lines

Page numbers

Page 3 of 12

Headers & footers

HEADER

main content region

FOOTER

Components stay inspectable

The rendered page and the test structure come from the same rows, columns, and components.

Component	Placement behavior	Practical note
Text / rich text	Measures wrapping inside the available column width.	Use auto rows for variable-length body copy.
Images / codes	Scale from the cell dimensions and local offsets.	Reserve clear space around scannable content.
Tables	Compute row height from their cell contents.	Good for dense facts, ledgers, and compatibility matrices.

Layout Model

Rows and columns on a 12-column grid, resolved to exact points on the PDF page.

Default denominator

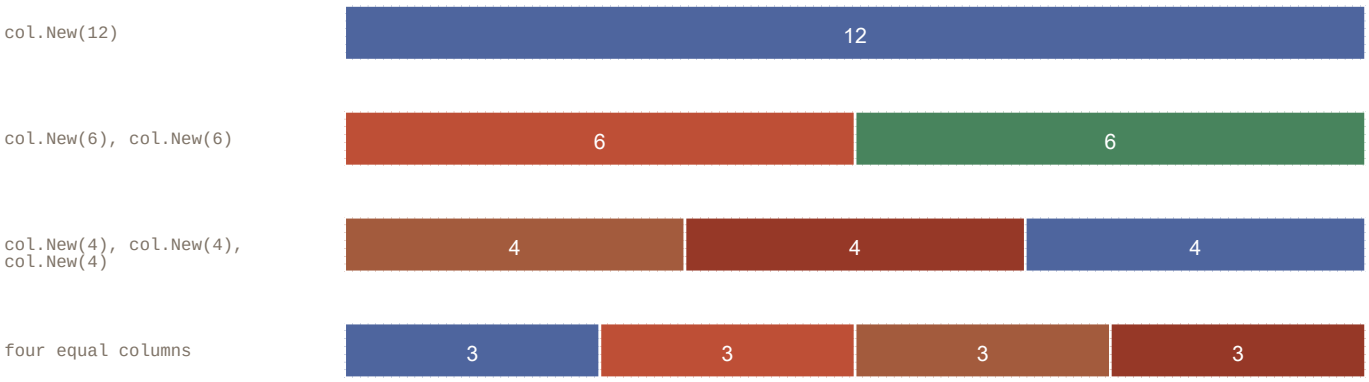
Paper defaults to a 12-unit grid.

Column widths

col.New(3) gets 3/12 of the row width.

Auto height

row.New() measures child content before rendering.



How placement works

Paper measures rows before rendering and creates new pages when the useful area is full.

1. Compute useful page area after margins, header, and footer.
2. Measure each row. Rows that overflow move to a new page.
3. Normalize column units against MaxGridSize.

Case	Behavior	Why it matters
Exact fit	Column units add to the configured grid size.	The row fills the useful page width.
Underflow	Unused grid units remain blank space.	Useful for indents, side notes, and sparse layouts.
Auto height	Largest child height defines the row.	Variable text and tables can remain deterministic.

Nested rows

Components can contain translated HTML or tables inside normal columns.

Auto rows

Measured content determines height before drawing begins.

Manual rows

Fixed heights keep reports and form regions predictable.

Page flow

Rows that do not fit continue on the next page with header and footer.

HTML Pipeline

HTML in, structured Paper rows out, then rendered through the same PDF provider.

Full document

paper.FromHTML returns a generated PDF document from one HTML string.

Append rows

m.AddHTML parses HTML into rows using the active Paper configuration.

Column fragment

htmlcomponent.New wraps translated rows as a regular component.

HTML support snapshot

The translator maps common HTML and CSS features onto Paper primitives.

Input	Rendered as	Notes
Headings, paragraphs, inline tags	Text and rich text runs	Supports bold, emphasis, links, colors, and spacing.
display:flex	Rows and columns	Flex widths quantize to the configured grid size.
table, border, background	Table cells and cell styles	Useful for reports, invoices, and rich document fragments.

Stage	Input	Paper primitive
Parse	DOM nodes and CSS declarations	Intermediate block tree
Translate	Blocks, flex, tables, inline runs	Rows, columns, text, rich text, tables
Render	Measured Paper rows	PDF cells on the page

Inline content

Paragraphs, headings, strong/emphasis, links, and colored text become text or rich text runs.

Block layout

Flex rows and common block spacing are mapped onto Paper rows and grid columns.

Document fit

Translated rows use the same page measurement, break, header, and footer behavior as hand-built rows.

HTML Rendered Output

This fragment is parsed through pkg/components/html and rendered as Paper rows.

Rendered HTML Fragment

Paper translates supported tags and CSS into the same row, column, text, table, and style primitives used elsewhere in this PDF.

Blocks headings, paragraphs, lists

Layout tables and flex rows

Styles colors, borders, spacing

HTML	Paper output
<p>, ,	rich text runs
display:flex	grid columns
table	table component cells

Rendered feature	Visible result	Backed by
Panel styling	Border, radius, padding, and background.	Cell styles
Flex KPI row	Three equal columns with gaps.	Grid columns
HTML table	Header row, borders, and data rows.	Table component

Why this page matters

The HTML example is not a screenshot. It is parsed into Paper components and rendered by the same provider pipeline as the rest of the document.

What to inspect

The panel border, flex-style KPI row, table header, cell borders, and text wrapping all come from translated HTML and CSS.

Testing and Metrics

Inspect the component tree before rendering, then track generation behavior after each run.

doc.Tree() - deterministic snapshot

```
Document a4 portrait
+-- Header
|   +-- Row [12]
+-- Row [8,4]
|   +-- Text "Invoice" ok
|   +-- QRCode 88x88 ok
+-- Table 4 rows
+-- Footer PageNumber
```

paper.Metrics() - last run

18 ms generation time	7 components
1 page	42 kb output size

throughput 1,240 docs/sec

Deterministic snapshots

Assert structure without image diffs. Rows, columns, tables, and codes all appear in the tree.

Generation timing

Optional metrics expose build phases, add-row timings, page creation, and output size.

Visual verification

Render the final PDF pages when layout or assets change, then keep the source as normal Go.

Use Cases

Built for the documents Go services generate on demand and at volume.

PDF

Invoices

totals, tax, QR pay

PDF

Reports

tables, charts, pages

PDF

Forms

checks, fields, signs

PDF

Labels

barcodes, batches

PDF

Certificates

signatures, seals

PDF

Contracts

clauses, appendices

Production APIs

Return bytes, base64 payloads, or files from the same document object.

Document automation

Generate customer-facing PDFs from queue workers and service jobs.

HTML migration

Reuse existing markup while moving toward typed component layouts.

Audit-friendly

Keep the generated structure testable and repeatable across releases.

Surface	What it shows	Typical output
Components	Explicit rows, columns, and typed primitives.	Operational PDFs and forms.
HTML	Markup translated into Paper rows.	Rich fragments and imported content.
Document	Headers, footers, page numbers, metadata, and bytes.	Files, streams, base64, and merged PDFs.

Generated PDF Example

A single invoice-style page using the same visual language as the showcase hero.

Paper Labs, Inc.

120 Foundry Street, Suite 4 / Brooklyn, NY 11222 / hello@paperlabs.dev

Invoice

NO. INV-2026-0481

Billed to

Northwind Trading Co. / Accounts Payable

Issue date

June 8, 2026 / Due July 8, 2026

Terms

Net 30 / PO-77-4521

Currency

USD ($) / Account NW-00231

Description	Qty	Unit price	Amount
Document API - Team plan	1	$480.00	$480.00
Rendering credits	12k	$0.02	$240.00
Priority support	1	$120.00	$120.00
Custom font embedding	3	$30.00	$90.00

Payment details

Wire transfer or ACH within 30 days. Reference the invoice number on all remittances.

Subtotal	$930.00
Volume discount (5%)	-$46.50
Tax (NY 8.875%)	$78.41
Total due	$961.91

Avdo S.

Authorized - Paper Labs

☒ Goods and services received in full

☒ Amount verified against PO-77-4521

☐ Recurring billing authorized

Install Paper

Ship documents that render exactly right.

Add Paper to your Go service and generate production PDFs in a single import.

```
$ go get github.com/avdoseferovic/paper
```

Docs

Read examples for HTML, components, tables, codes, and generated output.

GitHub

Use the repository examples as executable documentation for your service.

MIT License

Build PDFs in Go without adding a browser runtime to production jobs.